

QB 05: Web Technologies & Development — Question Bank with Answer Keys

Course: Web Technologies & Web Development (Theory + Lab)

Target: VTU MCA Semester 1 / Semester 2 (IPCC)

Scheme: VTU 2022 / 2024 Scheme

Contents: HTML5 Semantic Layouts, CSS3 Flexbox/Grid, ES6+ JavaScript, DOM Manipulation, Async/Await & Fetch API, 15 MCQs with Explanations

Syllabus Outline (VTU 5 Modules)

- **Module 1:** HTML5 Core, Semantic Tags (`<header>` , `<nav>` , `<article>` , `<section>` , `<footer>`), Media Elements (`<audio>` , `<video>`), Forms, Input Types, Validation Attributes.
- **Module 2:** CSS3 Fundamentals, Selectors, Box Model, Flexbox, CSS Grid Layout, Media Queries for Responsive Design, Transitions, and Animations.
- **Module 3:** JavaScript Essentials, Scopes (`var` , `let` , `const`), Hoisting, Closures, Higher-Order Functions (`map` , `filter` , `reduce`), ES6+ Arrow Functions, and Classes.
- **Module 4:** Document Object Model (DOM), Element Selection & Manipulation, Event Handling, Event Bubbling & Capturing, Web Storage API (`localStorage` , `sessionStorage`).
- **Module 5:** Asynchronous JavaScript: Callbacks, Promises, `async` / `await` , Fetch API, JSON Serialization, RESTful Architecture, and HTTP Status Codes.

SECTION 1: High-Yield MCQs with Answer Key

Q1. Which HTML5 tag represents independent, self-contained content that could be distributed in an RSS feed?

- A) `<section>`
- B) `<article>`
- C) `<aside>`
- D) `<div>`

Answer: B

Explanation: `<article>` specifies self-contained content (like a blog post, news story, or forum post) that makes sense on its own.

Q2. What is the difference between `let` and `var` in JavaScript?

- A) `var` is block-scoped; `let` is function-scoped
- B) `let` is block-scoped and does not hoist to the top with an initial value; `var` is function-scoped
- C) `let` cannot be reassigned
- D) There is no difference

Answer: B

Explanation: `let` is block-scoped `{ }` and resides in a "Temporal Dead Zone" before declaration; `var` is hoisted and scoped to the enclosing function.

Q3. In CSS Flexbox, which property aligns flex items along the cross-axis (perpendicular to the main axis)?

- A) `justify-content`
- B) `align-items`
- C) `flex-direction`
- D) `flex-wrap`

Answer: B

Explanation: `justify-content` aligns items along the main axis; `align-items` controls alignment along the cross axis.

Q4. What does the triple equals operator (`===`) test in JavaScript?

- A) Value equality with type coercion
- B) Strict equality: both value and data type must match without type conversion
- C) Reference memory address only
- D) Object inheritance

Answer: B

Explanation: `==` performs implicit type coercion (`'5' == 5` is true), whereas `===` performs strict comparison without conversion (`'5' === 5` is false).

Q5. Which method converts a JavaScript object into a JSON string?

- A) `JSON.parse()`
- B) `JSON.stringify()`
- C) `Object.toJSON()`
- D) `JSON.encode()`

Answer: B

Explanation: `JSON.stringify(obj)` serializes a JS object into a JSON string; `JSON.parse(str)` parses a JSON string back into an object.

Q6. In the DOM event model, what is the default order of event propagation?

- A) Event Capturing (outside $\rightarrow$ inside)
- B) Event Bubbling (target $\rightarrow$ ancestors)
- C) Random propagation
- D) Immediate halt

Answer: B

Explanation: By default, events bubble upward from the innermost target element through parent ancestors unless `useCapture = true` is set.

Q7. What HTTP status code denotes a successful resource creation on the server?

- A) 200 OK
- B) 201 Created
- C) 204 No Content
- D) 301 Moved Permanently

Answer: B

Explanation: `201 Created` indicates that the request succeeded and resulted in the creation of a new resource (standard response for successful `POST` requests).

Q8. What happens when calling `event.preventDefault()` inside a form submit handler?

- A) Stops the event from bubbling up the DOM
- B) Prevents the browser's default behavior (e.g., page reload or automatic navigation)
- C) Clears all input fields
- D) Disables JavaScript execution

Answer: B

Explanation: `event.preventDefault()` suppresses the native browser action (such as submitting an HTTP form causing a page reload).

Q9. What does the `async` keyword before a function guarantee?

- A) The function will run on a separate CPU thread
- B) The function will always return a Promise
- C) The function cannot contain loops
- D) The function blocks the event loop

Answer: B

Explanation: An `async` function implicitly wraps its return value in a resolved `Promise`.

Q10. What is the storage limit of HTML5 `localStorage` compared to cookies?

- A) Cookies: 4 KB; localStorage: ~5 MB to 10 MB
- B) Cookies: 5 MB; localStorage: 4 KB
- C) Both are identical (100 KB)
- D) localStorage expires on browser close

Answer: A

Explanation: Traditional cookies are limited to 4 KB and sent with every HTTP header. `localStorage` holds 5–10 MB of client-side key-value pairs with no expiration date.

 SECTION 2: Short Answer Concepts (4–6 Marks)

Q11. Explain Closures in JavaScript with a code example and practical use case.

Answer:

Definition:

A **closure** is the combination of a function bundled together with references to its surrounding lexical environment. In JavaScript, an inner function always retains access to the variables of its outer (enclosing) function, even after the outer function has finished execution and returned.

```
function createCounter(initialValue = 0) {
  let count = initialValue; // Private variable trapped in closure

  return {
    increment: function() {
      count++;
      return count;
    },
    decrement: function() {
      count--;
      return count;
    },
    getCount: function() {
      return count;
    }
  };
}

const counter = createCounter(10);
console.log(counter.increment()); // 11
console.log(counter.increment()); // 12
console.log(counter.decrement()); // 11
// console.log(counter.count);    // Undefined! Encapsulated private state!
```

Practical Use Cases:

1. **Data Encapsulation / Private Variables:** Emulating private properties before private class fields (`#field`) existed.
2. **Factory Functions & Currying:** Creating configured functions (e.g., logger functions, event listeners with presets).

Q12. Compare `Promise` and `async/await` syntax in JavaScript.

Answer:

| Feature | Promises (`.then()` / `.catch()`) | `async / await` (ES2017) | `---|---|---` | **Syntax** | Method chaining using callbacks inside `.then()` and `.catch()` | Synchronous-looking procedural style | | **Error Handling** | Handled using `.catch(error => { ... })` | Handled using standard `try { ... } catch (error) { ... }` blocks | | **Readability** | Can lead to "Promise chaining hell" with multiple sequential dependent calls | Clean, linear, and readable | | **Under the Hood** | Foundation of modern async JS | Syntactic sugar built directly on top of Promises |

```
// Promise syntax:
fetch('https://api.github.com/users/octocat')
  .then(res => res.json())
  .then(data => console.log(data.login))
  .catch(err => console.error(err));

// async/await syntax:
async function getGithubUser() {
  try {
    const res = await fetch('https://api.github.com/users/octocat');
    const data = await res.json();
    console.log(data.login);
  } catch (err) {
    console.error(err);
  }
}
```

SECTION 3: VTU Model Practical Questions (10–12 Marks)

Q13. [VTU Model QP - Module 4: Form Validation with Regex]

Create an HTML5 Registration Form containing: Name, USN, Email, Password, and Phone Number. Write JavaScript to validate that:

1. USN follows the VTU format (e.g., `1RV24MC001` or `1MS24MC045`).
2. Password must contain at least 8 characters, at least one uppercase letter, one digit, and one special symbol.
3. Phone number must be exactly 10 digits.
4. Display friendly error messages dynamically below the respective input field.

Answer:

```

<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <title>VTU MCA Registration Form Validation</title>
  <style>
    body { font-family: Arial, sans-serif; background: #f4f6f8; padding: 20px; }
    .form-card { max-width: 450px; margin: auto; background: white; padding: 25px; border: 1px solid #ccc; }
    .form-group { margin-bottom: 15px; }
    label { display: block; font-weight: bold; margin-bottom: 5px; }
    input { width: 100%; padding: 10px; border: 1px solid #ccc; border-radius: 4px; box-sizing: border-box; }
    .error-msg { color: #d9534f; font-size: 0.85em; margin-top: 4px; display: block; }
    .submit-btn { width: 100%; background: #007bff; color: white; padding: 12px; border: none; }
    .submit-btn:hover { background: #0056b3; }
  </style>
</head>
<body>

<div class="form-card">
  <h2>VTU MCA Registration</h2>
  <form id="regForm" novalidate>
    <div class="form-group">
      <label for="usn">VTU USN:</label>
      <input type="text" id="usn" placeholder="e.g., 1RV24MC001">
      <span id="usnError" class="error-msg"></span>
    </div>

    <div class="form-group">
      <label for="email">Email Address:</label>
      <input type="email" id="email" placeholder="student@vtu.ac.in">
      <span id="emailError" class="error-msg"></span>
    </div>

    <div class="form-group">
      <label for="phone">Phone Number (10 Digits):</label>
      <input type="tel" id="phone" placeholder="9876543210">
      <span id="phoneError" class="error-msg"></span>
    </div>

    <div class="form-group">
      <label for="password">Password:</label>
      <input type="password" id="password" placeholder="Min 8 chars, 1 upper, 1 number, 1 special char">
      <span id="passError" class="error-msg"></span>
    </div>

    <button type="submit" class="submit-btn">Register</button>
  </form>
</div>

<script>
document.getElementById('regForm').addEventListener('submit', function(e) {
  e.preventDefault(); // Prevent page reload
  let isValid = true;

```

```

// 1. Validate USN (Format: 1-4 alphanumeric characters e.g., 1RV24MC001)
const usn = document.getElementById('usn').value.trim();
const usnRegex = /^[1-4][A-Z]{2}\d{2}[A-Z]{2}\d{3}$/i;
const usnError = document.getElementById('usnError');
if (!usnRegex.test(usn)) {
    usnError.textContent = "Invalid VTU USN (format: e.g., 1RV24MC001)";
    isValid = false;
} else {
    usnError.textContent = "";
}

// 2. Validate Email
const email = document.getElementById('email').value.trim();
const emailRegex = /^[^\s@]+@[^\s@]+\.[^\s@]+$/;
const emailError = document.getElementById('emailError');
if (!emailRegex.test(email)) {
    emailError.textContent = "Please enter a valid email address.";
    isValid = false;
} else {
    emailError.textContent = "";
}

// 3. Validate Phone Number (10 digits starting with 6-9)
const phone = document.getElementById('phone').value.trim();
const phoneRegex = /^[6-9]\d{9}$/;
const phoneError = document.getElementById('phoneError');
if (!phoneRegex.test(phone)) {
    phoneError.textContent = "Phone number must be exactly 10 digits starting with 6-9.";
    isValid = false;
} else {
    phoneError.textContent = "";
}

// 4. Validate Password (Min 8 chars, 1 uppercase, 1 digit, 1 special symbol)
const password = document.getElementById('password').value;
const passRegex = /^(?=.*[A-Z])(?=.*\d)(?=.*[!@#$%^&*()_+])[A-Za-z\d!@#$%^&*()_+]{8,}$/;
const passError = document.getElementById('passError');
if (!passRegex.test(password)) {
    passError.textContent = "Password requires min 8 chars, 1 uppercase, 1 number, and 1 sp";
    isValid = false;
} else {
    passError.textContent = "";
}

if (isValid) {
    alert("Registration Successful! Form verified with zero validation errors.");
    this.reset();
}
});
</script>

```



```
</body>  
</html>
```

Q14. [VTU Model QP - Module 2: Responsive Layout with CSS Flexbox and Grid]

Write HTML and CSS code to design a modern responsive card grid for an MCA Course Portal displaying 3 courses (Cloud Computing, Cyber Security, DevOps). The layout must show 3 columns on desktops, 2 columns on tablets, and 1 column on mobile phones.

Answer:

```
<!DOCTYPE html>
<html lang="en">
<head>
  <meta charset="UTF-8">
  <meta name="viewport" content="width=device-width, initial-scale=1.0">
  <title>MCA Responsive Courses Grid</title>
  <style>
    * { box-sizing: border-box; margin: 0; padding: 0; }
    body { font-family: 'Segoe UI', Tahoma, Geneva, Verdana, sans-serif; background: #0f172a; color: #f1f5f9; }

    .header { text-align: center; margin-bottom: 40px; }
    .header h1 { font-size: 2.2rem; color: #38bdf8; }
    .header p { color: #94a3b8; margin-top: 8px; }

    /* Modern CSS Grid with Responsive Breakpoints */
    .course-grid {
      display: grid;
      grid-template-columns: repeat(3, 1fr); /* 3 columns for Desktop */
      gap: 25px;
      max-width: 1200px;
      margin: 0 auto;
    }

    .card {
      background: #1e293b;
      border: 1px solid #334155;
      border-radius: 12px;
      padding: 25px;
      display: flex;
      flex-direction: column;
      justify-content: space-between;
      transition: transform 0.3s ease, box-shadow 0.3s ease;
    }

    .card:hover {
      transform: translateY(-8px);
      box-shadow: 0 12px 24px rgba(56, 189, 248, 0.2);
      border-color: #38bdf8;
    }

    .card-icon { font-size: 40px; margin-bottom: 15px; }
    .card h2 { font-size: 1.4rem; color: #f1f5f9; margin-bottom: 10px; }
    .card p { color: #94a3b8; font-size: 0.95rem; line-height: 1.6; margin-bottom: 20px; }

    .enroll-btn {
      background: linear-gradient(135deg, #0284c7, #2563eb);
      color: white;
      padding: 10px 18px;
      border-radius: 6px;
      text-align: center;
      text-decoration: none;
      font-weight: bold;
      display: inline-block;
    }
```

```

    }

    /* Tablet Media Query (max-width: 900px) → 2 columns */
    @media (max-width: 900px) {
        .course-grid {
            grid-template-columns: repeat(2, 1fr);
        }
    }

    /* Mobile Media Query (max-width: 600px) → 1 column */
    @media (max-width: 600px) {
        .course-grid {
            grid-template-columns: 1fr;
        }
    }
}
</style>
</head>
<body>

<div class="header">
    <h1>VTU MCA Specializations</h1>
    <p>Master the in-demand skills for the Cloud, Cyber & DevOps Era</p>
</div>

<div class="course-grid">
    <div class="card">
        <div>
            <div class="card-icon">☁️</div>
            <h2>Cloud Architecture</h2>
            <p>Architect scalable, resilient infrastructure on AWS & Azure with VPC, IAM, a
        </div>
            <a href="#" class="enroll-btn">Explore Cloud</a>
        </div>

    <div class="card">
        <div>
            <div class="card-icon">🔒</div>
            <h2>Cybersecurity</h2>
            <p>Master defensive security, OWASP top 10 web vulnerabilities, cryptography, a
        </div>
            <a href="#" class="enroll-btn">Explore Cyber</a>
        </div>

    <div class="card">
        <div>
            <div class="card-icon">⚙️</div>
            <h2>DevSecOps Automation</h2>
            <p>Automate CI/CD pipelines using Docker, Kubernetes orchestration, Git, and Te
        </div>
            <a href="#" class="enroll-btn">Explore DevOps</a>
        </div>
    </div>

```

```
</body>  
</html>
```

Key Takeaway Checklist for Exam Day

- [] Memorize the exact regular expression syntax for email, phone, and password patterns.
- [] Understand CSS Flexbox properties: `flex-direction`, `justify-content` (main axis), `align-items` (cross axis).
- [] Remember the syntax difference between `localStorage.setItem('key', 'value')` and `sessionStorage`.
- [] Remember that `async/await` uses standard `try { ... } catch (err) { ... }` blocks for error handling.